Group B

Experiment No.:1

Write a program to scheme a few activation functions that are used in neural networks

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
SignatureofCourse Teacher	

Aim:

To study and implement various activation functions used in neural networks and understand their behavior through graphical representation.

Objective:

- To understand the role of activation functions in neural networks.
- To implement commonly used activation functions such as Sigmoid, Tanh, ReLU, Leaky ReLU, and Softmax.
- To visualize and compare the output of different activation functions.
- To analyze their practical applications in machine learning models.

Theory:

In Artificial Neural Networks, activation functions play a crucial role in determining whether a neuron should b activated or not. They introduce non-linearity into the model, enabling neural networks to learn complex patterns from data.

Without activation functions, a neural network behaves like a simple linear model, regardless of the number of layers.

1. Sigmoid Function

- Formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output range: (0, 1)
- Used in binary classification problems

2.Tanh Function

- Formula:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Output range: (-1, 1)

Zero-centered, better than sigmoid in many cases

3. Leaky ReLU

Formula:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{if } x \leq 0 \end{cases}$$

- Solves the "dead neuron" problem of ReLU

4.ReLU (Rectified Linear Unit)

- **Formula:** f(x)=max(0,x)
- Most widely used activation function
- Efficient and reduces computation time

5.Softmax Function

Formula:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

- Converts outputs into probability distribution
- Used in multi-class classification

Conclusion:

Activation functions help neural networks learn complex patterns by introducing non-linearity. Different functions like Sigmoid, ReLU, and Softmax are used for different tasks to improve model performance and accuracy.

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA credited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:2

Write a program to show back propagation network for XOR function with binary input and output

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
SignatureofCourse Teacher	

Aim:

To implement a backpropagation neural network to learn and simulate the XOR function with binary inputs and outputs.

Objectives:

- To understand the working of the backpropagation algorithm.
- To implement a neural network for solving the XOR problem.
- To train the network using forward and backward propagation.
- To analyze how weights are updated to minimize error.

Theory:

A Artificial Neural Network is a computational model inspired by the human brain. It consists of layers of neurons: input layer, hidden layer, and output layer.

The XOR (Exclusive OR) function is a non-linear problem, meaning it cannot be solved using a simple linear model. Therefore, a neural network with at least one hidden layer is required.

XOR Truth Table

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Backpropagation Algorithm

Backpropagation is a supervised learning algorithm used to train neural networks. It works in two main steps:

1. **Forward Propagation**
   - Inputs are passed through the network.
   - Output is calculated using weights and activation functions.
2. **Backward Propagation**
   - Error is calculated as the difference between actual and predicted output.

- The error is propagated backward.
- Weights and biases are updated to reduce error.

3.Activation Function

The **Sigmoid function** is commonly used:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

It converts input values into a range between 0 and 1.

The backpropagation process involves three main steps:

1. Forward Pass

- Input values are fed into the network.
- Each neuron computes a weighted sum:

$$z = \sum (w \cdot x) + b$$

The result is passed through an activation function (like sigmoid).

- Output is generated.

---

2. Error Calculation

- The difference between actual output and predicted output is calculated:

$$Error = Actual - Predicted$$

---

3. Backward Pass (Backpropagation)

- The error is propagated backward from output layer to hidden layer.
- Weights are updated using gradient descent:

$$w = w + \eta \cdot \delta \cdot x$$

where:

- η\etaη = learning rate
- δ\deltaδ = error gradient

Conclusion:

The backpropagation neural network successfully learns the XOR function, which is a non-linear problem. By using a hidden layer and updating weights through forward and backward propagation, the network minimizes error and produces accurate outputs. This demonstrates the importance of backpropagation in training neural networks for complex pattern recognition tasks.

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA credited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:3

Write a program for producing back propagation feed forward network

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
SignatureofCourse Teacher	

Aim:

- To design and implement a feed forward neural network using the backpropagation algorithm for learning and predicting outputs.

Objective:

- To understand the architecture of a feed forward neural network.
- To implement the backpropagation algorithm for training the network.
- To study forward propagation and backward propagation processes.
- To analyze how weights and biases are updated to minimize error.
- To evaluate the performance of the neural network.

Theory:

A Artificial Neural Network is a computational system inspired by biological neural networks. A Feed Forward Neural Network (FFNN) is the simplest type where data flows in one direction—from input layer to output layer—without loops.

Structure of Feed Forward Network

- **Input Layer** → Receives input data
- **Hidden Layer(s)** → Processes data using weights and activation functions
- **Output Layer** → Produces final result

Each neuron performs:

$$z = \sum (w \cdot x) + b$$

Activation Function

Activation functions introduce non-linearity. Commonly used:

- Sigmoid
- ReLU
- Tanh

Example (Sigmoid):

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Forward Propagation

- Input is passed through layers
- Weighted sum is calculated
- Activation function is applied

- Output is generated

Error Calculation

The difference between actual and predicted output:

$$Error = Actual - Predicted$$

Backpropagation Algorithm

Backpropagation is used to minimize error using gradient descent.

Steps:

1. Compute output using forward propagation
2. Calculate error
3. Propagate error backward
4. Update weights and biases

Weight update rule:

$$w = w + \eta \cdot \delta \cdot x$$

Where:

- η\etaη = learning rate
- δ\deltaδ = error gradient

Conclusion:

The feed forward neural network trained using backpropagation successfully learns the given input-output relationship. By iteratively updating weights and minimizing error, the model improves its prediction accuracy. This experiment demonstrates how backpropagation enables neural networks to solve complex problems efficiently.

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA credited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:4

Write a program to demonstrate ART

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Aim:

To implement Adaptive Resonance Theory (ART) neural network for clustering and pattern recognition.

Objective:

- To understand the concept of ART neural networks.
- To implement clustering using ART algorithm.
- To study how vigilance parameter affects clustering.
- To analyze pattern recognition using unsupervised learning.

Theory:

Adaptive Resonance Theory (ART) is an unsupervised learning model used for clustering and pattern recognition. It was developed to solve the stability-plasticity dilemma, i.e., how a system can learn new information without forgetting old knowledge.

Step 1: Input Presentation

- A binary input pattern is applied to the network.
- Example:

  X=[1,0,1,0]X = [1, 0, 1, 0]X=[1,0,1,0]

Step 2: Category Selection

- The network compares input with all existing clusters.
- A matching score is calculated:

  Match=∣X∩W∣∣X∣Match = \frac{|X \cap W|}{|X|}Match=∣X∣∣X∩W∣

- The cluster with highest match is selected.

Step 3: Vigilance Test

- **T**he match score is compared with vigilance parameter (ρ):
  - If Match ≥ ρ → Accept cluster
  - If Match < ρ → Reject cluster

Step 4: Resonance (Learning)

- If the match satisfies vigilance:
  - Resonance occurs
  - Weights are updated:

    Wnew=X∩WoldW_{new} = X \cap W_{old}Wnew=X∩Wold
- This strengthens the cluster representation

Step 5: Reset Mechanism

- If match fails:
  - That cluster is temporarily ignored
  - Next best cluster is checked

Step 6: New Cluster Creation

- If no existing cluster matches:
    - A new cluster is created
    - Input pattern becomes its prototype

Main Components of ART

1. **Input Layer (F1 Layer)**
    - Receives input patterns
2. **Comparison Layer**
    - Compares input with stored patterns
3. **Recognition Layer (F2 Layer)**
    - Stores clusters (categories)
4. **Vigilance Parameter (ρ)**
    - Controls similarity threshold

Conclusion:

The ART neural network successfully clusters input patterns based on similarity. It adapts to new data without forgetting previous patterns, making it effective for real-time pattern recognition and unsupervised learning tasks.

Aim:

To implement the perceptron learning algorithm and visualize its decision region using Python.

Objectives:

- To understand the perceptron learning rule.
- To train a perceptron model on linearly separable data.
- To visualize the decision boundary graphically.
- To analyze classification performance of the perceptron.

Theory:

A Machine Learning perceptron is the simplest type of artificial neural network used for binary classification.

It is a single-layer feed-forward network that classifies input data into two classes using a linear decision boundary.

Perceptron Learning Rule

Weights are updated as:

$$w = w + \eta(t - y)x$$

$$b = b + \eta(t - y)$$

Where:

- η\etaη = learning rate
- ttt = target output
- yyy = predicted output

Working:

Step 1: Initialize Weights Randomly

- Assign small random values to weights w1,w2,...w_1, w_2, ...w1,w2,... and bias bbb.
- Example: w=[0.2,−0.1],b=0.0
  Learning rate η is also set (e.g., 0.1).

Step 2: Apply Input and Compute Output

- Take one input vector x=[x1,x2]
- Compute weighted sum:

  z=w·x+b

- Apply activation (step function):

  If Z > 0 → Output =1

  If Z< 0 → Output=0

Step 3: Compare with Target Output

- Compare predicted output yyy with actual target ttt.
- Calculate error:

  $$\text{Error}=t-y$$

Step 4: Update Weights Using Learning Rule

- If prediction is wrong, update weights and bias:

$$w = w + \eta(t-y)x$$

$$b = b + \eta(t-y)$$

If output is **too low**, increase weights

If output is **too high**, decrease weights

Step 5: Repeat Until Error is Minimized

- Repeat steps 2–4 for all training samples
- One complete pass = **1 epoch**
- Continue for multiple epochs until:
  - All points are correctly classified, OR
  - Error becomes very small

Conclusion:

The perceptron learning algorithm effectively classifies linearly separable data by iteratively updating weights based on error. It learns a decision boundary that separates different classes, demonstrating a simple yet powerful approach to binary classification in neural networks.